

Infotact Technical Internship Program: Software Engineering Project Specifications and Evaluation Criteria

The following document establishes the technical requirements, architectural directives, and sprint-based development roadmaps for three distinct software engineering projects. Designed for execution within a fast-paced technology environment in Bengaluru, Karnataka, these specifications serve as comprehensive blueprints for engineering interns entering the infotact startup ecosystem. The documentation provides structured frameworks intended to guide the development of scalable, cloud-native applications across three high-growth industry niches: Retail and E-commerce, Travel and Hospitality, and Healthcare Technology.

The primary directive of this documentation is to standardize the engineering process, ensuring that all deliverables align with enterprise-grade architectural patterns, rigorous version control protocols, and robust security standards. The Indian developer ecosystem has evolved into the largest globally, with a profound shift toward deep-tech startups, cloud-native architectures, and artificial intelligence integration. Consequently, engineering interns must operate at a professional level, demonstrating proficiency in continuous integration, algorithmic problem-solving, and full-stack system design.

This document outlines the requisite tasks, technological paradigms, and evaluation criteria for successful internship completion. It is imperative to note that the assessment of these projects relies heavily on the transparency and consistency of the development lifecycle. The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.

Project 1: Omnichannel Retail Point of Sale (POS) and Inventory Management System

Executive Problem Statement

In the rapidly digitizing retail landscape, offline businesses transitioning to digital models frequently encounter systemic bottlenecks due to fragmented legacy software. As multi-location retailers scale their operations, they face severe latency issues under large data sets, leading to inaccurate inventory synchronization across brick-and-mortar stores, warehouses, and online storefronts. Disconnected systems necessitate manual workflows that result in stockouts, overselling, and disjointed pricing strategies. Conventional Point of Sale (POS) platforms often degrade in performance as the database swells with historical transaction data and expansive catalog records, rendering them unsuitable for high-velocity retail environments.

The objective of this project is to architect a cloud-native, omnichannel POS and Inventory Management System that unifies physical and digital retail operations into a single source of truth. The platform must process transactions with high throughput, maintain real-time inventory

parity across multiple geographic nodes, and provide actionable business intelligence through a centralized dashboard. This solution will serve as the core "Offline to Online Transformation" package for traditional retail enterprises.

Business Objectives and Key Performance Indicators

To ensure the solution delivers tangible enterprise value, the development lifecycle must be oriented around specific operational metrics and algorithmic efficiencies. The primary engineering objective is the realization of an omnichannel synchronization engine capable of achieving sub-second latency in updating inventory across all connected sales channels immediately following a transaction. The architecture must prioritize system reliability, maintaining a high-availability infrastructure capable of functioning during localized network degradation and synchronizing offline POS transactions seamlessly once connectivity is restored.

Data integrity stands as a paramount concern. The system must eliminate discrepancy rates between physical stock and digital records through strict ACID (Atomicity, Consistency, Isolation, Durability) compliant database transactions, ensuring that no concurrent operations lead to race conditions or phantom reads. The performance expectations require API response times to remain below 200 milliseconds at the 95th percentile, with inventory synchronization delays not exceeding 1.5 seconds across distributed nodes. Furthermore, the overarching infrastructure must guarantee a system uptime of 99.9% or higher, mirroring enterprise-grade Service Level Agreements.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Store Cashier	Fast checkout processes, barcode scanning integration, and a highly intuitive, low-latency interface.	Executes daily retail transactions, processes refunds, applies dynamic promotional codes, and registers new walk-in customers within a sub-second response environment.
Inventory Manager	Real-time stock visibility across all nodes, low-stock algorithmic alerts, and	Monitors volumetric stock levels across multiple localized warehouses, initiates inter-store stock transfers, and reconciles

	automated purchase order generation.	physical inventory audits against digital ledgers.
System Administrator	Granular user access control, macro-level financial reporting, and comprehensive API management.	Configures multi-jurisdictional tax rules, manages multi-store hierarchies, and extracts predictive analytics regarding macroscopic consumer purchasing behavior.

Minimum Viable Product Specifications

The core functional requirements for the initial release emphasize operational stability, data integrity, and critical retail workflows. The foundation of the system relies on a Unified Product Catalog, which necessitates a centralized database structure supporting complex Product Information Management (PIM). This includes the capacity to handle multi-variant items distinguishing between size, color, and SKU, alongside hierarchical categorization and rule-based dynamic pricing.

The Omnichannel Order Management module functions as the routing engine, aggregating incoming orders from physical cashier terminals and digital storefronts. This engine must compute the optimal fulfillment center based on the mathematical optimization of geographic distance and localized stock availability. Complementing this is the Real-Time POS Terminal, a web-based register interface engineered for rapid item addition, cart modification, localized tax calculation, and multi-modal payment integration encompassing cash, credit, and digital wallets. Finally, the Automated Inventory Engine serves as the tracking module that decrements stock atomically upon order confirmation, utilizing basic forecasting algorithms to predict reorder points based on historical sales velocity.

Architectural Directives and Technology Stack

The application must follow a decoupled, headless architecture to ensure maximum scalability, isolating the frontend presentation layer from the backend business logic.

Component	Technology	Architectural Rationale

Frontend Application	React.js, Tailwind CSS	Enables dynamic UI development with reusable components and highly responsive POS interfaces adaptable across diverse tablet and desktop hardware profiles.
Backend Services	Node.js, Express.js, TypeScript	Provides a highly concurrent, asynchronous runtime environment ideal for I/O heavy operations and real-time transaction processing, while TypeScript ensures rigorous compile-time type safety.
Primary Database	MongoDB via Mongoose ORM	A NoSQL document structure that offers immense flexibility. Mongoose serves as the standard Object Data Modeling (ODM) library, letting you create a structured schema and offering an easy way to communicate with the database.
Caching Layer	Redis	Operates as a blazing-fast, in-memory data structure store to drastically reduce database read loads for frequently accessed data, such as product catalogs and active promotional rules.
Deployment Pipeline	Docker, Vercel/AWS, GitHub Actions	Ensures strict environment parity from local development to production through containerization. Automated CI/CD pipelines govern testing and deployment.

Four-Week Engineering Roadmap

The project execution is strictly divided into four weekly sprints, demanding rigorous adherence to the delivery schedule and continuous integration practices.

Week 1: System Architecture, Schema Design, and Authentication

The inaugural week demands the establishment of the foundational infrastructure. Interns must initiate the GitHub repository, configure branch protection rules requiring pull request reviews, and initialize the monorepo structure housing both the React.js frontend and Express backend. The local development environment must be containerized using Docker to instantiate MongoDB and Redis instances, ensuring cross-platform consistency.

Following environment setup, the intern must design and implement the document schema utilizing the Mongoose ODM. Core collections must include entities for Users, Stores, Products, Inventory Ledgers, Orders, and Order Line Items. The schema design must explicitly define references, cascading deletion logic, and indexing strategies tailored for high-frequency search queries. Concurrently, the intern must implement a secure, stateless authentication mechanism utilizing JSON Web Tokens (JWT) and bcrypt for cryptographic password hashing. A robust Role-Based Access Control (RBAC) middleware layer must be established to differentiate permissions strictly between Cashiers, Managers, and System Administrators.

Week 2: Core API Development and Inventory Logic

The second week focuses on backend business logic and data mutation. The intern is required to develop RESTful API endpoints governing the Product Catalog, encompassing all Create, Read, Update, and Delete (CRUD) operations. To ensure optimal performance over large datasets, the implementation must feature cursor-based pagination and advanced search functionality utilizing MongoDB's native full-text search and indexing capabilities.

Subsequently, the intern will construct the core Inventory Engine, specifically the transaction processing endpoint. This critical infrastructure must verify transactional integrity using MongoDB multi-document transactions to ensure that financial ledger updates, stock decrementing, and order creation succeed or fail as a single atomic unit, thereby absolutely preventing data corruption or phantom inventory scenarios. To mitigate latency, the intern must integrate Redis caching for the product catalog, implementing intelligent cache invalidation strategies to ensure that administrative updates to product pricing immediately reflect across all active cashier sessions. Unit tests utilizing Vitest or Jest must be written for all critical business logic paths to verify transactional integrity.

Week 3: Frontend Framework Integration and POS Interface

Week three transitions the focus toward the user-facing application. The intern will initialize the React.js frontend, configuring a global state management solution such as Redux Toolkit or Zustand. This state manager must flawlessly handle the active POS cart, the current authenticated user session, and detect offline/online network status to support resilient retail operations.

The primary deliverable for this sprint is the POS terminal User Interface. This interface must be heavily optimized for speed, supporting keyboard shortcuts and rapid barcode-simulated product searches. The intern must implement the cart calculation algorithm on the client side, computing the final price by aggregating unit prices, applying dynamic discount matrices, and

incorporating localized taxation rates. The frontend components must then be wired to the backend APIs developed in Week 2, implementing robust error handling and optimistic UI updates to enhance perceived system performance. Responsive design utilizing Tailwind CSS is mandatory to ensure cross-device compatibility.

Week 4: Stabilization, Testing, and Deployment Pipeline Setup

The final week is dedicated to system stabilization, quality assurance, and production deployment. The intern must conduct comprehensive end-to-end (E2E) testing, utilizing frameworks like Cypress or Playwright to simulate concurrent cashier transactions, actively hunting for race conditions or database deadlocks.

Following bug resolution, the intern must finalize the Continuous Integration and Continuous Deployment (CI/CD) pipeline utilizing GitHub Actions. These automated workflows must execute code linting, static type-checking, and the full unit test suite upon every Pull Request. The deployment must be automated, pushing the frontend to Vercel and the containerized backend to a cloud provider such as AWS or Render. Finally, the intern must generate comprehensive API documentation using Swagger or OpenAPI, accompanied by a detailed README file elucidating environment setup, architectural decisions, and schema diagrams. The repository commit history must reflect a continuous, daily engineering effort spanning the entire four-week period, demonstrating an iterative approach to software construction.

Project 2: Integrated Food Delivery and Dine-Out Hospitality Platform

Executive Problem Statement

The modern food technology and hospitality industry suffers from significant digital fragmentation. Consumers are routinely forced to navigate disparate, isolated applications for food delivery, table reservations, and live event ticketing, leading to acute application fatigue and a disjointed user experience. Conversely, restaurant partners struggle with exorbitant third-party platform commission fees and highly disjointed operational dashboards that complicate inventory and table management.

Furthermore, existing platforms frequently suffer from a systemic lack of detailed, qualitative user data. Consumers often leave generic star ratings rather than rich, descriptive text reviews, effectively depriving recommendation algorithms of the nuanced natural language data required to generate highly personalized, accurate suggestions. The directive for this project is to architect an integrated aggregator application that unifies food delivery, table reservations, and event discovery into a singular, cohesive ecosystem. The platform must feature an advanced, gamified review system to incentivize high-quality user feedback, operating alongside a complex geospatial search engine.

Business Objectives and Key Performance Indicators

The strategic vision for this platform centers on Ecosystem Integration, consolidating multiple hospitality verticals to exponentially increase the Average Revenue Per User (ARPU) and total user session duration. The platform must dramatically amplify user feedback generation by increasing both the volume and the average character length of consumer text reviews. This will be achieved through the implementation of structured psychological incentives and AI-assisted writing prompts that lower the cognitive friction of leaving detailed feedback.

Operational efficiency for merchant partners is a critical KPI. The system must provide restaurants with a unified command dashboard to seamlessly manage delivery queues, physical table availability, and dynamic pricing models. From an engineering perspective, the system's success will be quantified by specific metrics: achieving a high Review Conversion Rate (the ratio of completed orders to submitted detailed reviews), maintaining search latency below 200 milliseconds for complex geospatial queries, and reducing the overall Customer Acquisition Cost (CAC) through algorithmic cross-selling between the dining and event verticals.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
End Consumer	Seamless geospatial discovery of nearby merchants, transparent fee structures, and reliable, real-time delivery tracking.	Executes multi-parameter searches for venues, schedules synchronized table reservations, tracks live GPS delivery status, and submits gamified reviews to accrue platform loyalty points.
Restaurant Partner	Unified digital management of online orders and physical tables, lower commission paradigms, and actionable data insights.	Modifies menu availability dynamically, accepts or rejects incoming delivery dispatches via WebSockets, and monitors sentiment analysis derived from customer reviews.

Delivery Courier	Algorithmic routing optimization, transparent compensation visibility, and highly intuitive navigation interfaces.	Receives instantaneous delivery dispatch alerts, utilizes integrated geospatial navigation to transit between merchants and consumers, and tracks daily earnings.
-------------------------	--	---

Minimum Viable Product Specifications

The foundational feature of this platform is the Geospatial Merchant Discovery engine. This location-based search infrastructure allows users to pinpoint restaurants and events utilizing advanced filtering vectors such as cuisine type, aggregate rating, and precise kilometric distance, executed via highly optimized spatial indexing.

The application requires a Unified Cart and Checkout system characterized by complex state-management capabilities. This allows users to fluidly transition between queuing delivery orders and locking in table reservations, integrated with a secure, simulated payment gateway handling transaction states. Post-transaction, the Gamified Review Engine activates. This module prompts users to leave detailed reviews, calculating a dynamic reward score based on text length and keyword relevance, and awarding digital loyalty points accordingly. Finally, the platform must feature a Real-Time Order State Machine, a WebSocket-driven tracking system that pushes synchronous updates to the consumer, merchant, and courier as an order transitions from "Accepted" to "In Transit" and finally "Delivered".

Architectural Directives and Technology Stack

Given the necessity for rapid geographic queries and real-time bidirectional communication, the technology stack diverges significantly from standard relational architectures.

Component	Technology	Architectural Rationale
Web Frontend	React.js	Facilitates web development with a unified codebase, ensuring consistent user experiences across web environments while accelerating feature iteration.

Backend API Services	Node.js, Express.js	Express.js provides a lightweight backend framework that simplifies server-side development, enabling developers to create RESTful APIs and handle HTTP requests efficiently.
Primary Database	MongoDB (Atlas)	A NoSQL document structure is highly optimal for handling highly variable menu schemas and executing blazing-fast <code>\$geoNear</code> geospatial aggregation pipelines.
Real-time Communication	WebSockets (Socket.io)	Absolutely essential for pushing live order status updates and streaming courier GPS coordinates to client devices without relying on aggressive, resource-heavy HTTP polling.
Cloud Infrastructure	AWS (EC2, S3)	Ensures scalable compute resources and provides highly durable object storage (S3) for restaurant imagery and user-generated media content.

Four-Week Engineering Roadmap

Interns must adhere to this structured timeline to successfully navigate the complexities of geospatial databases and real-time event streaming.

Week 1: Geospatial Architecture and Application Scaffolding

The project initiates with defining the JSON data contracts for all REST APIs and establishing the core repository. The intern must configure a MongoDB Atlas cluster, establishing secure connection protocols and environment variable management.

The primary engineering task is designing the NoSQL document schemas. The intern must architect collections for Users, Restaurants, Menu Items, and Orders. Crucially, the Restaurant schema must incorporate GeoJSON point data structures. The intern is required to implement the 2dsphere geospatial indexing required to enable high-performance proximity-based search functionality. By the end of the week, the authentication layer utilizing JWT must be fully

operational, featuring secure endpoints for user registration, login, and profile modification. All backend code must adhere to strict Node.js and Express.js linting rules and be committed to the repository with clear, descriptive messaging.

Week 2: Merchant Discovery, Complex Cart Logic, and WebSockets

Week two centers on data retrieval and real-time communication. The intern will build the restaurant discovery APIs, implementing the complex backend logic to execute `$geoNear` aggregation pipelines. These pipelines must return paginated lists of restaurants, mathematically sorted by a combination of geographic distance and user rating.

Concurrently, the intern must develop the complex cart state management architecture on the frontend. The system must gracefully handle edge cases, such as explicitly preventing users from simultaneously adding items from multiple distinct geographic restaurants into a single delivery cart. The week concludes with the initialization of the WebSocket server. The intern must establish an event-driven architecture capable of broadcasting order status mutations (e.g., `ORDER_PREPARING`, `COURIER_ASSIGNED`) to the respective client applications in real-time, ensuring state synchronization across multiple disparate devices.

Week 3: Transactional Checkout, Review Gamification, and AI Prompts

The third week tackles financial transaction logic and user engagement systems. The intern must implement the checkout flow, integrating a mock payment gateway that reliably updates the database order status upon successful charge simulation.

The most complex task of this sprint is developing the Gamified Review Engine. The intern must implement the backend parsing logic to analyze incoming text reviews. The system must programmatically allocate points based on an algorithm that weighs word count, keyword density, and the presence of uploaded media. Furthermore, the intern must integrate a basic AI assistance layer—utilizing a Node.js NLP library or an external API—to suggest descriptive keywords to the user based on their specific order history, thereby artificially lowering the cognitive friction required to write a high-quality review.

Week 4: Operational Dashboards, Query Optimization, and Deployment

The final week is dedicated to merchant tooling and system optimization. The intern will develop the simplified Merchant Dashboard interface, providing restaurant partners with a secure portal to toggle item availability, accept incoming web-socket orders, and visualize daily revenue aggregates.

Rigorous integration testing is mandatory. The intern must write tests to verify that WebSocket connections gracefully handle reconnections and network drops without losing payload data. Furthermore, MongoDB queries must be aggressively optimized using `explain()` execution plans to guarantee that 2dsphere indexes are correctly utilized and full-collection scans are eliminated. The project concludes with deployment configuration using Docker containers, the implementation of GitHub Actions to enforce automated testing prior to merging Pull Requests, and the finalization of comprehensive system documentation.

Project 3: Telemedicine and Electronic Health Records (EHR) Application

Executive Problem Statement

The global healthcare sector is undergoing a massive, structural paradigm shift toward decentralized, digital-first patient care. However, traditional clinical workflows remain heavily reliant on siloed, on-premise Electronic Health Records (EHR) systems that inhibit data portability, restrict interoperability, and frustrate both medical practitioners and patients. Furthermore, the lack of deeply integrated telehealth infrastructure forces clinics to rely on disjointed, consumer-grade video conferencing tools. These external tools frequently fail to meet the stringent data privacy and encryption regulations (such as HIPAA analogs) required for handling highly sensitive medical consultations.

This project demands the end-to-end engineering of a secure, cryptographically compliant Telemedicine and EHR platform. The system must facilitate a seamless continuum of patient care, encompassing algorithmic appointment scheduling, secure peer-to-peer video consultations, encrypted management of patient medical histories, and automated, verifiable prescription generation.

Business Objectives and Key Performance Indicators

The uncompromising priority for this application is Data Security and Privacy. The system architecture must ensure absolute data confidentiality through robust, military-grade encryption applied both at rest in the database and in transit across the network, decisively mitigating the risk of unauthorized access to Protected Health Information (PHI).

Operational workflow integration is the secondary objective. The platform must drastically reduce administrative friction for medical staff by seamlessly linking the appointment scheduling matrix with dynamically generated, secure teleconsultation rooms and real-time clinical documentation interfaces. The platform must remain highly accessible, providing a responsive, intuitive interface that accommodates patients with varying levels of digital literacy, while guaranteeing low-latency data transmission during critical video consultations. Success will be measured strictly: the system must maintain an uptime of 99.8% or higher, establish WebRTC video connections in under three seconds, and pass automated vulnerability scanning with zero critical security flaws.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow

Patient	Frictionless appointment booking, highly secure access to personal medical history, and reliable, low-latency video connectivity.	Interfaces with the scheduling matrix to book slots, uploads previous diagnostic reports securely, joins WebRTC video rooms via encrypted tokens, and downloads verifiable digital prescriptions.
Medical Practitioner	Efficient chronological schedule management, instantaneous retrieval of patient history, and streamlined clinical note-taking interfaces.	Approves incoming appointments, reviews longitudinal patient health data streams, conducts high-definition video consultations, and generates cryptographic, immutable prescriptions.
Clinic Administrator	Macroscopic system management, regulatory compliance oversight, and billing/insurance integration tracking.	Manages doctor availability rosters, oversees platform access logs for compliance auditing, and extracts anonymized, aggregated clinical analytics.

Minimum Viable Product Specifications

The application foundation is the Secure EHR Module. This highly structured database system must store longitudinal patient records encompassing demographics, vital signs, allergy histories, and chronological diagnoses. To meet compliance standards, all fields containing sensitive PHI must utilize AES-256 encryption at the application layer prior to database insertion.

The Smart Scheduling Engine serves as the logistical core. This complex calendaring system must handle localized time-zone conversions, absolutely prevent double-booking through algorithmic collision detection, and interface with external services to issue automated email or SMS reminders. The clinical delivery mechanism relies on Encrypted Telehealth Rooms. The intern must integrate WebRTC technology to establish secure, peer-to-peer audio and video communication channels between patients and doctors, ensuring the data stream operates strictly within the browser environment without requiring external plugins. Concluding the consultation workflow is the Digital Prescription Generator, a module allowing doctors to input

medication details and generate standardized, unalterable PDF prescriptions containing embedded digital signatures and QR code verification mechanisms.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Frontend Web Application	React, TypeScript, Material-UI	Delivers a highly structured, accessible user interface. TypeScript enforces rigid data structures when parsing and transmitting complex medical data payloads.
Backend API Services	Node.js, Express.js	Employed to build the server-side logic of the application, leveraging middleware support for tasks like authentication and error handling.
Primary Database	MongoDB	A NoSQL database used for storing data in JSON-like format, perfectly matched for the JavaScript ecosystem.
Video Infrastructure	WebRTC & Socket.io (Signaling)	WebRTC enables low-latency, peer-to-peer encrypted video streams. Socket.io handles the critical initial signaling (SDP exchange) required to negotiate the peer connection through firewalls.

Security & Auditing	JWT, AES-256, Audit Logging Middleware	Guarantees all PHI is encrypted and ensures that every system action (read/write) is securely, immutably logged to maintain regulatory compliance.
--------------------------------	--	--

Four-Week Engineering Roadmap

The execution of this project requires extreme attention to detail regarding data handling and security protocols.

Week 1: Compliance Architecture, Schema Design, and Cryptographic Foundations

The project begins with the establishment of a secure GitHub repository, mandating strict branch protection rules that require peer reviews for all code merges. The intern must provision the MongoDB database and architect the NoSQL EHR schema, encompassing tables for Patients, Doctors, Appointments, Medical Records, and Prescriptions.

The critical engineering task for week one is the implementation of the security foundation. After establishing standard JWT-based authentication, the intern must develop field-level encryption logic within the backend Mongoose models. This ensures that sensitive patient data is mathematically encrypted utilizing AES-256 before it ever touches the database disk. Furthermore, the intern must develop comprehensive Audit Logging middleware. Every single API request that accesses or mutates PHI must be intercepted and logged with a timestamp, user ID, IP address, and an exact descriptor of the action taken, creating an immutable trail for compliance audits.

Week 2: EHR API Development and Algorithmic Scheduling Engine

Week two focuses on data accessibility and logistical algorithms. The intern will construct the core CRUD APIs for the Electronic Health Records. These endpoints require rigorous input validation utilizing robust schemas (e.g., Zod) to absolutely prevent NoSQL injection or cross-site scripting (XSS) vulnerabilities.

Subsequently, the intern must engineer the Smart Scheduling Engine. The core of this engine is a collision detection algorithm that must mathematically verify availability before confirming an appointment. For any proposed appointment interval, the system must execute an overlap check against all existing intervals for that specific doctor, rejecting the payload if a temporal collision exists. Finally, the frontend React application must be initialized and integrated with these APIs, constructing intuitive calendaring interfaces and highly structured forms for clinical data entry.

Week 3: WebRTC Telehealth and Signaling Integration

The third week introduces real-time video streaming complexities. The intern must establish a dedicated Node.js/Socket.io signaling microservice. This server's sole responsibility is handling the exchange of Session Description Protocol (SDP) payloads and ICE candidates, which are mathematically necessary to negotiate the peer-to-peer WebRTC connection across various NATs and firewalls.

Following the signaling infrastructure, the intern must implement the WebRTC logic within the React frontend. This involves developing the consultation room UI, managing both local and remote media streams by requesting explicit browser camera and microphone permissions. The engineering must account for graceful degradation, ensuring the application handles network drops or peer connection failures elegantly. Finally, the scheduling engine must be linked to the telehealth module, generating unique, cryptographically secure room tokens for each appointment, guaranteeing that patients and doctors can only gain access to the video room at the exact scheduled time.

Week 4: Prescription Generation, Security Audits, and Final Deployment

The final sprint is dedicated to document generation and platform hardening. The intern must develop the Digital Prescription module, creating backend logic that utilizes libraries like PDFKit in Node.js to generate highly formatted, professional PDF documents based on the physician's pharmacological inputs. These documents must include a secure cryptographic hash or QR code to allow external pharmacies to verify the prescription's authenticity.

Prior to deployment, rigorous security testing is mandatory. The intern must utilize static code analysis tools and dependency vulnerability scanners (such as GitHub Advanced Security or Dependabot) to ensure absolutely no exploitable flaws exist within the application stack. The application must then be containerized using Docker and deployed to a secure cloud environment. The intern must finalize the repository documentation, explicitly detailing the cryptographic algorithms utilized and providing exact steps to reproduce the local development environment.

Universal Engineering and Version Control Guidelines

To simulate a rigorous, enterprise-grade development environment indicative of top-tier product companies in Bengaluru, strict adherence to modern software engineering practices is an absolute requirement. The core tenet of this internship program is continuous, transparent, and high-quality contribution. Engineering is not merely the final codebase; it is the iterative, documented process of problem-solving.

Mandatory Evaluation Protocol

The evaluation of the intern's performance and the final project submission is governed by a strict, non-negotiable directive regarding version control consistency.

The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.

A project submitted with a compressed commit history—for example, uploading the entire codebase in a massive, monolithic push during the final week or on the final day—will result in immediate disqualification. Professional engineering is a continuous process of iteration, and the version control history must accurately reflect daily progress, architectural pivots, code refactoring, and debugging efforts.

GitHub Operational Standards and Security Practices

To pass the technical evaluation, interns must internalize and strictly adhere to the following Git and GitHub operational standards, which mirror industry best practices for secure software supply chains.

1. Commit Frequency, Granularity, and Semantic Messaging

Commits act as the immutable ledger of an engineer's thought process. Interns are expected to commit code frequently, breaking down complex features into logical, highly isolated units of work. A minimum of three to five meaningful commits per active development day is the expected baseline. Monolithic commits containing thousands of lines of code spanning multiple disparate features are strictly prohibited.

Furthermore, commit messages must communicate the exact intent and context of the change, not merely the files affected. Interns must utilize the Conventional Commits specification, utilizing structured prefixes such as `feat:`, `fix:`, `refactor:`, or `docs:` to maintain a highly readable repository history.

2. Branching Strategy and Pull Request (PR) Etiquette

Direct commits to the `main` or `master` branch are entirely forbidden. Interns must utilize a structured Git Flow or GitHub Flow methodology. All new development must occur on isolated feature branches. Code is merged into the main branch exclusively via Pull Requests. Each PR must contain a comprehensive description outlining the specific problem solved, the architectural approach taken, and instructions for testing the feature. Interns are required to conduct a documented self-review on their own Pull Requests, utilizing GitHub's comment feature to explain complex lines of code or architectural decisions prior to executing the merge.

3. Continuous Integration and Cryptographic Security (CI/CD)

To demonstrate proficiency in modern DevOps practices, all repositories must implement automated workflows utilizing GitHub Actions. A GitHub Action must trigger on every Pull Request, automatically installing dependencies, executing the unit test suite, and failing the build if any tests fail.

Security hygiene is paramount. Repositories must enforce the principle of least privilege. The `GITHUB_TOKEN` permissions within workflow files must be explicitly scoped to read-only access unless write access is strictly required for a specific deployment task. Furthermore,

hardcoding API keys, database credentials, or secret tokens within the codebase is grounds for immediate failure. All sensitive data must be injected via secure Environment Variables and managed strictly through GitHub Secrets or OpenID Connect (OIDC) integrations.

By strictly enforcing these comprehensive specifications, the program ensures that deliverables transcend mere functional prototypes, transforming into robust, scalable, and highly maintainable software architectures indicative of elite engineering standards.